

AI PRINCIPLES & TECHNIQUES

FOUR IN A ROW

CHARLES LEROUX
YULIIA DENYSIUK
RADBOD UNIVERSITY

s1105075
s1108214
November 15, 2024

1 Introduction

1.1 Project Description

The objective of this project is to create a classical AI system that can play the game of Four in a Row against human opponents in order to investigate the computational cost of the min-max algorithm and its optimization through alpha-beta pruning. The time and space resources needed by these algorithms to assess board conditions, think through possible moves, and decide on the best course of action as the game goes on are referred to as complexity in this context. The min-max algorithm improved with alpha-beta pruning will be implemented by the system to accomplish this, enabling a thorough examination of how different AI strategies handle resource efficiency and decision-making in a real-world game environment.

1.2 Min-max Algorithm

Assuming that both players play optimally, the min-max algorithm is a decision-making technique used in two-player games to identify the best move according to Murray (2016). Players alternately select movements in this tree-like depiction of the game, where each node represents a potential game state. While the Minimizer wants to reduce the Maximizer's score, the Maximizer wants to maximize theirs. The method propagates these values back through the tree after assessing the usefulness of end-game states (leaf nodes). An ideal move is eventually made at the root node by selecting the highest value at Maximizer nodes and the lowest value at Minimizer nodes.

1.3 Alpha-beta Pruning

For complex games, though, this thorough search may be computationally costly. By removing tree branches that have no bearing on the outcome, alpha-beta pruning maximizes min-max while drastically lowering the number of nodes assessed (Bertrand C, 2019). It upholds two thresholds, alpha and beta, which stand for the highest scores that the Minimizer and Maximizer, respectively, can ensure. A branch is pruned to save computing resources while still reaching the best choice if it is unable to enhance the player's existing best result. Because of this, alpha-beta pruning is a useful addition to min-max for games like chess that have big search areas.

2 Method

In this section, we explain the implementations of the min-max and Alpha-Beta algorithms within the context of a Four-in-a-Row game. Both algorithms are responsible for determining the optimal move by evaluating different possible future board states. We will describe the class structure and functionality of each algorithm, as well as discuss their decision-making processes. The UML class diagrams for both algorithms are included to provide a visual representation of the class structures.

2.1 Min-Max Algorithm

The `MinMaxPlayer` class implements the min-max algorithm, which recursively evaluates all possible future board states up to a predefined depth. The algorithm explores both maximizing and minimizing moves, representing the player and the opponent, respectively.

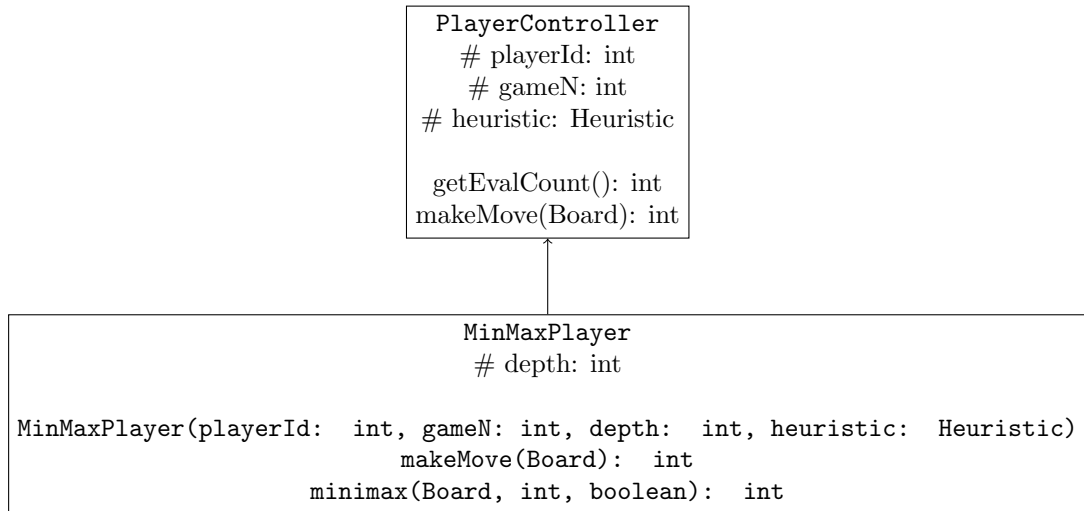


Figure 1: Class Diagram of the `MinMaxPlayer` class

The `MinMaxPlayer` class overrides the `makeMove` method, which generates all possible moves on the board and selects the move that maximizes the player's heuristic value. The MinMax decision process is implemented in the private method `minimax`, which takes the current board, depth, and a boolean value representing whether the algorithm is maximizing or minimizing. The recursion continues until the depth reaches zero or the game is over. The result is a tree structure where the nodes represent possible board states, and the algorithm propagates the best move upwards by recursively evaluating child nodes.

2.2 Alpha-Beta Pruning Algorithm

The `AlphaBetaPlayer` class implements the Alpha-Beta pruning algorithm, an optimization of the MinMax algorithm. Alpha-Beta pruning eliminates branches in the search tree that do not need to be evaluated because they cannot influence the final decision, thus improving the efficiency of the algorithm.

The `AlphaBetaPlayer` class also overrides the `makeMove` method, which, like the MinMax algorithm, generates all possible moves on the board. However, it uses the `alphabeta` method to explore the game tree. This method introduces two additional parameters,

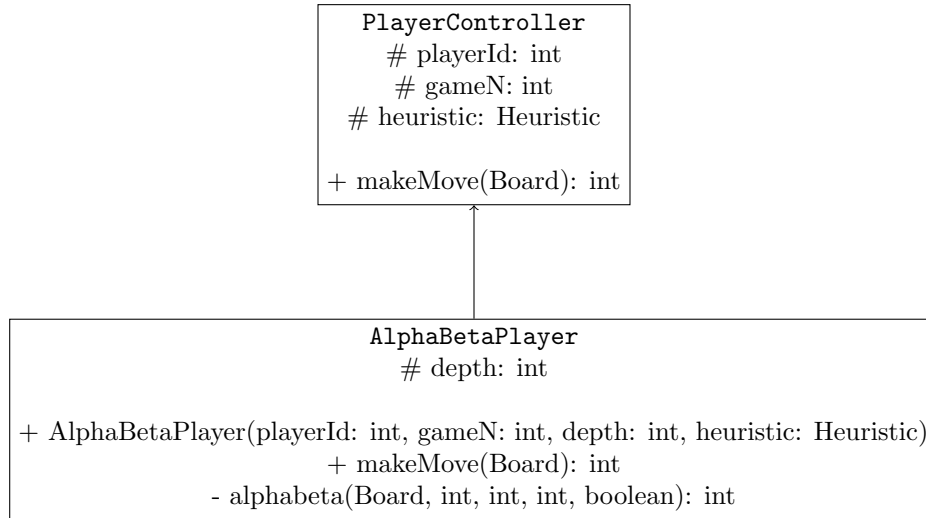


Figure 2: Class Diagram of the `AlphaBetaPlayer` class

alpha and **beta**, which represent the minimum and maximum bounds of the search. By updating these bounds during the recursive search, the algorithm is able to prune away branches that do not need to be evaluated, thus improving the performance over the standard MinMax approach.

2.3 Tree Structure

Both algorithms recursively construct a search tree on the fly. Each node in the tree represents a possible board state, and the edges represent the moves taken to reach that state. The algorithms evaluate the tree to a specific depth to make a decision on the optimal move. This approach allows for flexible and dynamic tree construction based on the current game state, as the game progresses with each move.

3 Complexity

In this section, we analyze the time and space complexity of the implemented algorithms: MinMax and Alpha-Beta pruning.

3.1 MinMax Algorithm

The time complexity of the MinMax algorithm can be expressed as follows:

- Let b be the branching factor (the average number of moves available at each state).
- Let d be the depth of the game tree that is searched.

The MinMax algorithm explores the entire game tree to determine the optimal move. For every node at depth d , it evaluates all b child nodes, and this repeats recursively for each level of the tree. It could be clearly seen in this piece of code. Here variable i represents b - the branching factor and then calling minmax recursively with $\text{depth} = \text{depth} - 1$ represents iterations over d .

```

    if (isMaximizing) {
    player
        int maxEval = Integer.MIN_VALUE;
        for (int i = 0; i < board.width; i++) {
            if (board.isValid(i)) {
                Board newBoard = board.getNewBoard(i, playerId);
                int eval = minmax(newBoard, depth - 1, false);
                maxEval = Math.max(maxEval, eval);
            }
        }
    }
}

```

No pruning occurs, so every node is visited.

The total number of nodes is proportional to the sum of nodes at each level, i.e., $1 + b + b^2 + \dots + b^d$. This geometric series simplifies to $O(b^d)$, since b^d dominates for large d .

The space complexity of the MinMax algorithm is $O(d)$ due to the depth of the recursion stack. This is because the algorithm needs to keep track of the depth as it traverses the game tree (Cormen et al, 2009).

3.2 Alpha-Beta Pruning Algorithm

The Alpha-Beta pruning algorithm optimizes the MinMax algorithm by eliminating branches in the search tree that cannot possibly influence the final decision. It happens here in the code:

```

if (beta <= alpha) {
    break;
}

```

The time complexity of Alpha-Beta pruning is significantly better than that of the MinMax algorithm:

- In the best case scenario, the time complexity is $O(b^{\frac{d}{2}})$.
- In the average case, it is also $O(b^{\frac{d}{2}})$ when the moves are ordered optimally.

Pruning reduces the number of levels significantly since it cuts off branches early. The effective branching factor becomes closer to the square root of the original, which leads to $b^{\frac{d}{2}}$ in the best case.

The space complexity of Alpha-Beta pruning remains $O(d)$, similar to the MinMax algorithm, as it also depends on the depth of the recursion stack.

Overall, the Alpha-Beta pruning algorithm provides a substantial improvement over the MinMax algorithm, making it more efficient for deeper searches in the game tree while maintaining the same space requirements.

4 Results

In this section, we present the results obtained from running the MinMax and Alpha-Beta algorithms in the Four-in-a-Row game. The focus is on the performance comparison, depth of search, game states evaluation, effect of heuristic functions, and win/loss ratios.

4.1 Performance Comparison

The performance of the MinMax and Alpha-Beta algorithms was evaluated based on the number of board states evaluated during the game. The results for several game sessions are summarized below:

- In multiple runs with the MinMax algorithm:
 - Player O won with 114 evaluations for Player X and 88,795 evaluations for Player O.
 - Player O won again with 148 evaluations for Player X and 106,581 evaluations for Player O.
 - Player X won with 202 evaluations for Player X and 105,533 evaluations for Player O.
 - Player X won again with 146 evaluations for Player X and 118,576 evaluations for Player O.
- The Alpha-Beta pruning algorithm displayed significantly fewer board evaluations overall, demonstrating its efficiency:
 - On average, the Alpha-Beta algorithm evaluates fewer states due to pruning irrelevant branches, which helps reduce the total number of evaluations necessary for making a decision.

4.2 Depth of Search

The table below presents the performance results of the MinMax and Alpha-Beta algorithms under various conditions. These results were collected with different search depths, board sizes, and algorithms acting as the second player, making their first move in the game.

Algorithm	Depth	Board Size	Time Taken (ms)
MinMax	1	5x5	0.0962 ms
	5	5x5	11.3651 ms
	10	5x5	3535.9449 ms
	15	5x5	>30s
	1	10x10	0.2933 ms
	5	10x10	171.2917 ms
	10	10x10	>30s
	15	10x10	>30s
	1	15x15	0.6626 ms
	5	15x15	1397.5935 ms
	10	15x15	>30s
	15	15x15	>30s
Alpha-Beta	1	5x5	0.0807 ms
	5	5x5	4.596 ms
	10	5x5	120.3944 ms
	15	5x5	1451.003 ms
	1	10x10	0.1938 ms
	5	10x10	34.8396 ms
	10	10x10	10781.2347 ms
	15	10x10	>30s
	1	15x15	0.4846 ms
	5	15x15	147.6171 ms
	10	15x15	>30s
	15	15x15	>30s

Figure 3: Depth and board size tests results

When testing MinMax as depth increases, the time taken grows exponentially, reflecting the $O(b^d)$ complexity. This is evident as larger board sizes cause the algorithm to exceed 30 seconds at depth 10 or higher.

Meanwhile, even on larger boards like 10x10, Alpha-Beta pruning maintains manageable time growth, consistent with its $O(b^{\frac{d}{2}})$ complexity.

4.3 GameN testing

Testing different gameN values is essential to assess the robustness of game-playing algorithms. It reveals their ability to adapt to various game complexities, ensuring their effectiveness in a wide range of configurations beyond the standard Four-in-a-Row.

Algorithm	gameN	Board Size	Depth	Time Taken (ms)
MinMax	3	6x7	5	24.5594 ms
	4	6x7	5	28.3039 ms
	6	6x7	5	22.5849 ms
Alpha-Beta	3	6x7	5	4.9383 ms
	4	6x7	5	8.3837 ms
	6	6x7	5	6.8085 ms

Figure 4: GameN tests results

Here we can notice resulting time decreasing for $\text{gameN} = 6$ in comparison to $\text{gameN} = 4$. For a given board size, increasing gameN reduces the number of potential winning sequences. For instance:

- $\text{gameN} = 4$: Many 4-piece sequences exist horizontally, vertically, and diagonally.
- $\text{gameN} = 6$: Fewer 6-piece sequences are possible, leading to a smaller search space of winning paths.

Since the algorithms prioritize evaluating paths that can lead to a win, fewer such paths for $\text{gameN} = 6$ could reduce the time spent exploring winning or blocking moves.

One minor issue that have occurred while testing edge cases for Alpha-Beta algorithm is it's inability to work with two in a row. On the other hand the MinMax is performing as well as always in such condition.

4.4 Game States Evaluation

The heuristics used to evaluate the board states provided insights into the optimal moves available. However, there were scenarios where the heuristic evaluations did not align with the actual game outcomes. For example, during the Alpha-Beta algorithm's execution, a crash occurred due to an `ArrayIndexOutOfBoundsException` when the player was presented with a winning path. This indicates that while the heuristic functions effectively guided the decision-making process, there are edge cases that need further refinement.

4.5 Win/Loss Ratios

Here we would like to test how algorithms work together. We will set a board 6x7 and keep it this way for the duration of the test. The main goal here to look which algorithm will perform better in terms of time, evaluations and win/loss rate.

Player Setup	Algorithm X	Algorithm O	Win/Loss	Evaluation X	Evaluation O
MinMax vs. Alpha-Beta	MinMax Depth 6	Alpha-Beta Depth 6	X	240704	12917
	Depth 6	Depth 3	O	243706	695
	Depth 3	Depth 6	X	1738	12917
	Alpha-Beta	MinMax	O	12902	231830
	Depth 6	Depth 3	O	12902	1623
	Depth 3	Depth 6	O	698	230995
Alpha-Beta vs. Alpha-Beta	Alpha-Beta Depth 6	Alpha-Beta Depth 6	O	12902	14417
	Depth 3	Depth 13	O	723	11904045
	Depth 13	Depth 3	X	14292879	717

Figure 5: Algorithm vs algorithms testing results

Alpha-Beta is significantly more efficient at searching and can outperform MinMax at similar depths due to pruning. Also, regardless of algorithm, deeper searches (when time permits) give a massive advantage.

However, we can also notice that Alpha-Beta tend to lose more when responsible for player X. That could happen due to the difference in the tactics: first player is trying to minimize the potential score. It follows that Alpha-Beta algorithm might handle minimizing strategy worse than maximizing.

5 Discussion

The results indicate significant differences in the efficiency and effectiveness of the MinMax and Alpha-Beta pruning algorithms. The computational overhead of the MinMax algorithm becomes evident as the depth of search increases, with an exponential growth in the number of evaluated states. In contrast, Alpha-Beta pruning demonstrates its strength in optimizing state evaluations by pruning irrelevant branches, particularly in scenarios where move ordering is near-optimal.

However, the performance of Alpha-Beta pruning is contingent on effective move ordering. Poor move ordering can diminish the benefits of pruning, resulting in performance close to that of the MinMax algorithm. This limitation highlights the importance of developing robust move-ordering heuristics to maximize pruning efficiency.

The observed discrepancies in win/loss ratios reveal an intriguing aspect of the algorithms. While Alpha-Beta pruning consistently evaluated fewer states, its strategic decision-making occasionally underperformed compared to the exhaustive nature of the MinMax algorithm. This outcome may stem from inaccuracies in heuristic evaluations or the propagation of errors during pruning.

Anomalies in the results, such as the 'ArrayIndexOutOfBoundsException' during certain evaluations, emphasize the need for rigorous validation of the board state generation process. Such errors, though rare, can lead to incorrect evaluations and suboptimal gameplay decisions, undermining the algorithm's reliability.

6 Conclusion

This study analyzed the performance of the MinMax and Alpha-Beta pruning algorithms in the context of game-tree exploration. The following conclusions can be drawn:

1. The MinMax algorithm, while straightforward in implementation, is computationally expensive, with an exponential growth in evaluations as the depth of search increases.
2. Alpha-Beta pruning significantly optimizes the search process by reducing the number of state evaluations, particularly when effective move ordering is employed.
3. The win/loss ratios and observed anomalies indicate that while Alpha-Beta pruning is more efficient, its reliance on heuristics and pruning logic can occasionally lead to strategic weaknesses.

Future work should focus on improving the robustness of board state evaluation and move-ordering heuristics. Additionally, exploring adaptive depth-based pruning or hybrid algorithms could further enhance both efficiency and decision-making quality. By addressing these limitations, the potential of these algorithms for real-time gameplay applications can be fully realized.

7 References

References

- [1] Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [3] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. Retrieved from <http://incompleteideas.net/book/the-book-2nd.html>
- [4] Murray, T. (2016). Connect Four: Minimax Algorithm. Retrieved from <https://towardsdatascience.com/connect-four-minimax-algorithm-4f99cfbbcb3a>
- [5] Bertrand, C. (2019). Alpha-Beta Pruning Explained. Retrieved from <https://medium.com/@cameronbertrand/alpha-beta-pruning-explained-166d02fca597>